



INSTALACIÓN Y CONFIGURACIÓN BÁSICAS DE UN SERVIDOR MULTIMEDIA

PARP204



24 DE NOVIEMBRE DE 2025
ANDRÉS VIZCAÍNO BAZAGA
DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

ÍNDICE

INTRODUCCIÓN	2
PRUEBA DE DLNA MEDIA STREAMING NATIVO DE WINDOWS	3
JELLYFIN SERVER	5
MINI-DLNA	8
CONCLUSIÓN.....	11

INTRODUCCIÓN

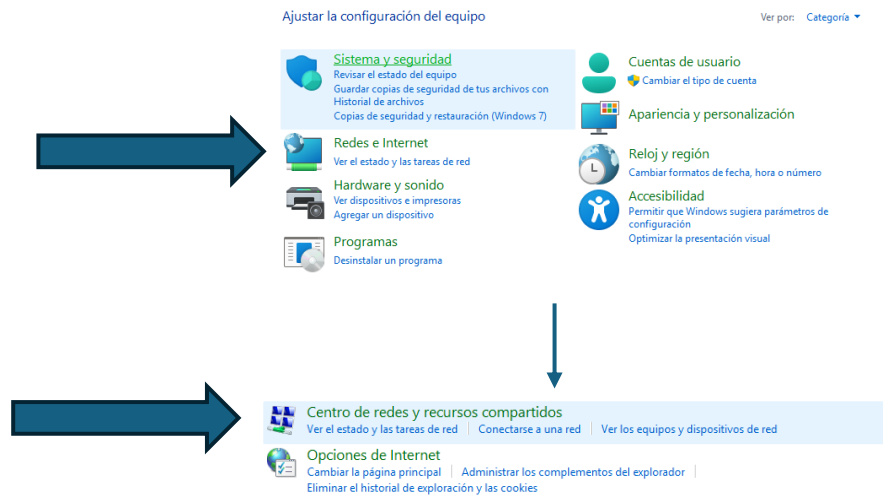
Para realizar esta práctica he utilizado un Windows 11 Pro, un Ubuntu Desktop y iPhone 15.

En el caso de Windows, el servidor que he instalado es: **Jellyfin Server**.

En el caso de Ubuntu Desktop, el servidor que he instalado es: **MiniDLNA**.

PRUEBA DE DLNA MEDIA STREAMING NATIVO DE WINDOWS

Lo primero que debemos hacer es irnos al panel de control de Windows, navegar a “Redes e Internet” ---> “Centro de redes y recursos compartidos”.

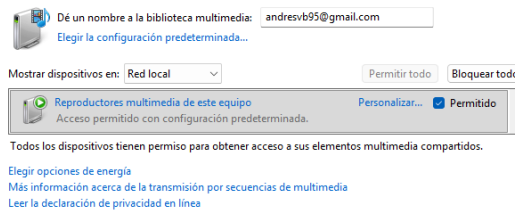


Una vez realizado este paso previo, debemos seleccionar “Opciones de Media streaming” y se hacemos clic en “Activar media streaming”.

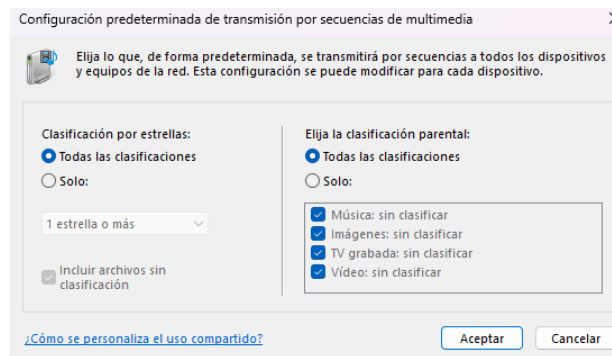


INSTALACIÓN Y CONFIGURACIÓN BÁSICAS DE UN SERVIDOR MULTIMEDIA

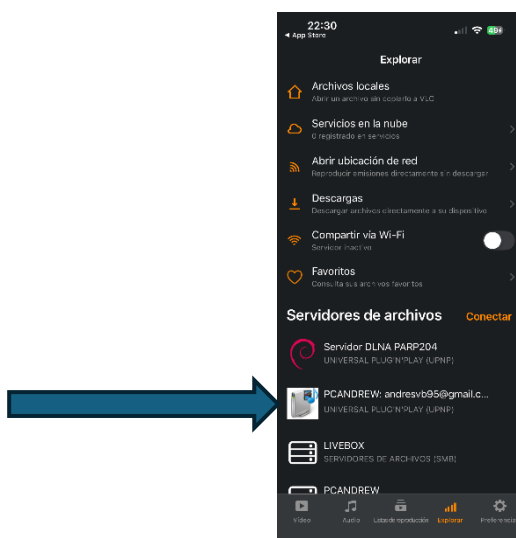
Elegir las opciones de transmisión por secuencias de multimedia para equipos y dispositivos



A continuación, revisamos y confirmamos los “*Ajustes por defecto*” de la biblioteca multimedia (clasificaciones y acceso de dispositivos).



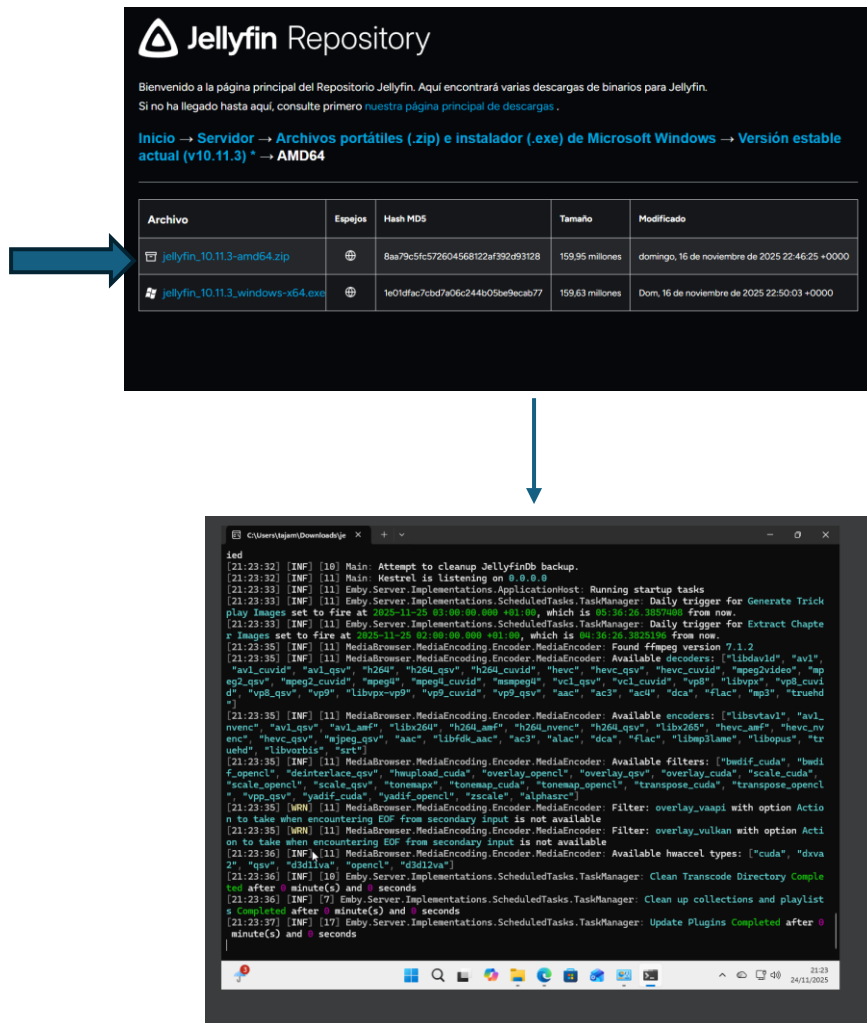
Por último, cogemos nuestro iPhone y con la app VLC Media Player, que es una app compatible con este servidor, miramos en nuestra red local si podemos acceder a nuestro servidor y observamos y que podemos acceder a él con éxito.



JELLYFIN SERVER

Lo primero que debemos hacer es descargar Jellyfin Server, y para que Windows nos deje hacer debemos desactivar el firewall. Si no hacemos este paso previo no vamos a poder descargarlo.

Una vez hecho, descargamos el archivo zip desde su página web y lo ejecutamos.



The image shows the Jellyfin Repository website with a table of download links. A blue arrow points from the 'jellyfin_10.11.3-amd64.zip' link to a terminal window below. The terminal window displays the startup logs of the Jellyfin server, showing that it is listening on port 8096.

Archivo	Español	Hash MD5	Tamaño	Modificado
jellyfin_10.11.3-amd64.zip		8aa79c5fc572604568122af392d93128	159.95 millones	domingo, 16 de noviembre de 2025 22:46:25 +0000
jellyfin_10.11.3-windows-x64.exe		1e01dfac7bd7a06c244b05be9ecab77	159.63 millones	Dom, 16 de noviembre de 2025 22:50:03 +0000

```

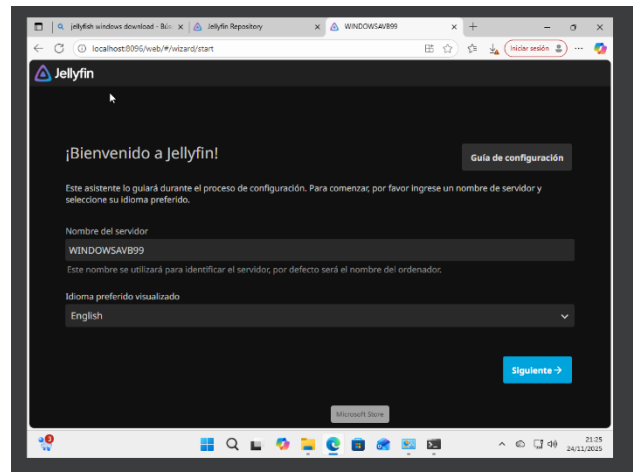
[21:23:32] [INF] [10] Main: Attempt to cleanup JellyfinDb backup.
[21:23:32] [INF] [11] Main: Kestrel is listening on 0.0.0.0
[21:23:33] [INF] [11] Emby.Server.Implementations.ApplicationHost: Running startup tasks
[21:23:33] [INF] [11] Emby.Server.Implementations.ScheduledTasks.TaskManager: Daily trigger for Generate Trick
play Images set to fire at 2025-11-26 03:00:00.000 +01:00, which is 00:36:26.8857668 from now.
[21:23:33] [INF] [11] Emby.Server.Implementations.ScheduledTasks.TaskManager: Daily trigger for Extract Chapte
r Images set to fire at 2025-11-26 03:00:00.000 +01:00, which is 00:36:26.8857668 from now.
[21:23:35] [INF] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Found ffmpeg version 7.1.2
[21:23:35] [INF] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Available decoders: ["libdav1d", "avi",
"av1_cuid", "av1_gsv", "h264", "h264_gsv", "h264_cuid", "hevc", "hevc_gsv", "hevc_cuid", "mpeg2vid", "mp
eg2_gsv", "mpeg2_cuid", "mpeg", "mpeg_cuid", "mpeg_gsv", "vc1_cuid", "vc1_gsv", "vp8", "libvpx", "vp8_cuid",
"vp8_gsv", "vp9", "libvpx-vp9", "vp9_cuid", "vp9_gsv", "aac", "ac3", "ac4", "dca", "flac", "mp3", "truehd
"]
[21:23:35] [INF] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Available encoders: ["libsvtav1", "av1_
nvenc", "av1_gsv", "av1_amf", "libx264", "h264_amf", "h264_nvenc", "h264_gsv", "libx265", "hevc_amf", "hevc_nv
enc", "hevc_gsv", "h265_gsv", "aac", "libfdk_aac", "ac3", "alac", "dca", "flac", "libmp3lame", "libopus", "tr
uehd", "libvorbis", "ac4"]
[21:23:35] [INF] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Available filters: ["bwdif_cuda", "bwdi
f_opengl", "deinterlace_gsv", "hwupload_cuda", "overlay_opengl", "overlay_gsv", "overlay_cuda", "scale_cuda",
"scale_opengl", "scale_gsv", "tonemap", "tonemap_cuda", "tonemap_opengl", "transpose_cuda", "transpose_opengl",
"vpp_gsv", "bwdif_cuda", "bwdif_opengl", "scale", "alphauc"]
[21:23:35] [WRN] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Filter: overlay_vaapi with option Acti
on to take when encountering EOF from secondary input is not available
[21:23:35] [WRN] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Filter: overlay_vulkan with option Acti
on to take when encountering EOF from secondary input is not available
[21:23:36] [INF] [11] MediaBrowser.MediaEncoding.Encoder.MediaEncoder: Available hwaccel types: ["cuda", "d3d11",
"qsv", "d3d11va", "opengl", "d3d12va"]
[21:23:36] [INF] [10] Emby.Server.Implementations.ScheduledTasks.TaskManager: Clean Transcode Directory Comple
ted after 0 minute(s) and 0 seconds
[21:23:36] [INF] [7] Emby.Server.Implementations.ScheduledTasks.TaskManager: Clean up collections and playlist
Completed after 0 minute(s) and 0 seconds
[21:23:37] [INF] [17] Emby.Server.Implementations.ScheduledTasks.TaskManager: Update Plugins Completed after 0
minute(s) and 0 seconds
  
```

Cuando lo ejecutamos, podemos observar que el servidor está corriendo perfectamente y que el mensaje clave es: ***“[INF] [21:23:32] Main: Kestrel is listening on 0.0.0.0:8096”***. Esto significa que el servidor web de Jellyfin (Kestrel) está escuchando y listo para ser accedido en el puerto 8096 en nuestro equipo.

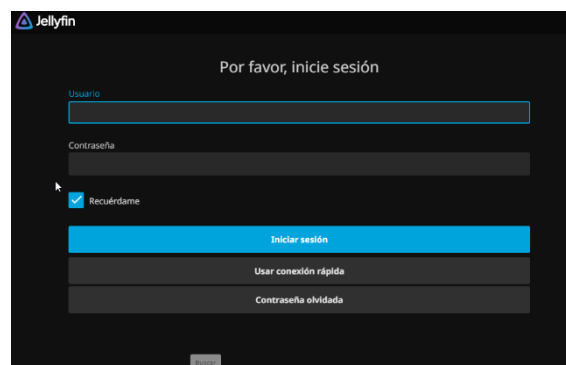
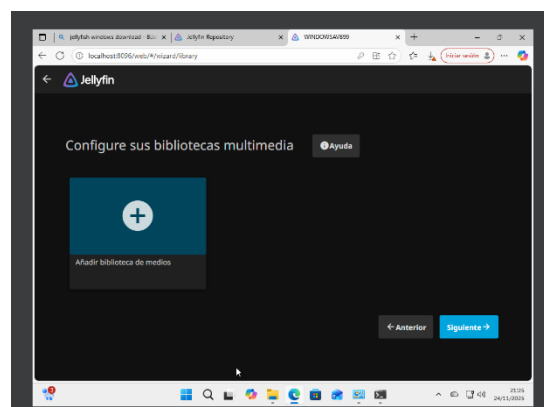
Por lo tanto, lo que debemos hacer a continuación es escribir la siguiente dirección en la barra:

- <http://localhost:8096>

INSTALACIÓN Y CONFIGURACIÓN BÁSICAS DE UN SERVIDOR MULTIMEDIA



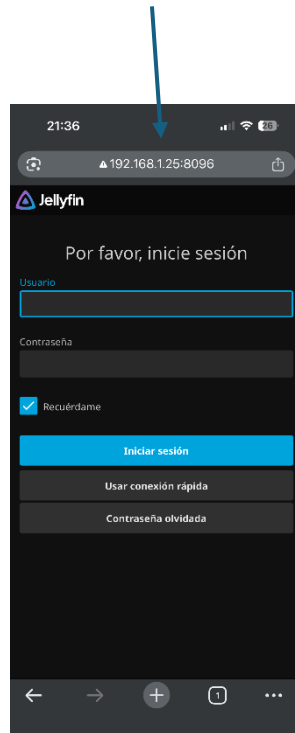
Realizamos las configuraciones necesarias y añadimos a nuestra biblioteca contenido para poder ser consumido. A continuación, tenemos nuestro servidor listo para poder acceder a él.



INSTALACIÓN Y CONFIGURACIÓN BÁSICAS DE UN SERVIDOR MULTIMEDIA

Si realizamos la práctica en una máquina virtual, debemos de ir a ajustes de red en el hardware de la máquina y debemos cambiar el modo del adaptador a “*Adaptador Puente*” para que pueda actuar nuestra máquina como un dispositivo independiente en nuestra red local.

Una vez hecho esto, realizamos la prueba en nuestro iPhone introduciendo en el navegador nuestra IP con el puerto 8096 que es el que hemos visto anteriormente.



MINI-DLNA

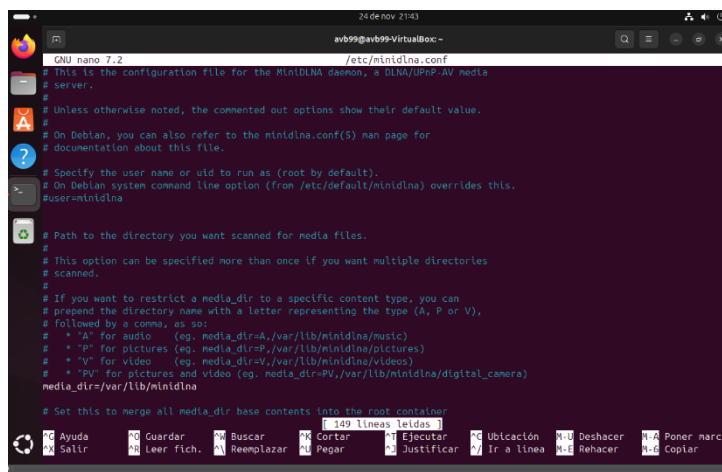
Lo primero que debemos hacer es instalar el paquete en la terminal, lo que requiere tener nuestra Máquina Virtual de Ubuntu en modo “*Adaptador Puente*” para que sea visible en nuestra red como hicimos en la anterior parte.

A continuación, instalamos MiniDLNA mediante el siguiente comando, asegurando la actualización previa de los repositorios:

- `sudo apt update.`
- `sudo apt install minidlna.`

Una vez instalado, debemos editar el archivo de configuración principal “`/etc/minidlna.conf`” usando el editor nano con permisos de administrador, para ello lo realizamos con el siguiente comando:

- `sudo nano /etc/minidlna.conf.`



```

GNU nano 7.2 /etc/minidlna.conf
# This is the configuration file for the MiniDLNA daemon, a DLNA/UPnP AV media
# server.
#
# Unless otherwise noted, the commented out options show their default value.
#
# On Debian, you can also refer to the minidlna.conf(5) man page for
# documentation about this file.
#
# Specify the user name or uid to run as (root by default).
# On Debian system command line option (from /etc/default/minidlna) overrides this.
#user=minidlna
#
# Path to the directory you want scanned for media files.
#
# This option can be specified more than once if you want multiple directories
# scanned.
#
# If you want to restrict a media_dir to a specific content type, you can
# prepend the directory name with a letter representing the type (A, P or V),
# followed by a comma, as so:
# * "A" for audio (eg. media_dir=A,/var/lib/minidlna/music)
# * "P" for pictures (eg. media_dir=P,/var/lib/minidlna/pictures)
# * "V" for video (eg. media_dir=V,/var/lib/minidlna/videos)
# * "PV" for pictures and video (eg. media_dir=PV,/var/lib/minidlna/digital_camera)
media_dir=/var/lib/minidlna
#
# Set this to merge all media_dir base contents into the root container
#

```

Dentro del archivo, realizamos dos modificaciones esenciales para que el servidor funcione y sea reconocido en la red:

1. **Configuración de Rutas Multimedia (media_dir):** Buscamos la sección de “`media_dir`” y añadimos la ruta donde se encuentran nuestros archivos de prueba dentro de la MV de Ubuntu, usando las letras clave para clasificarlos (V para vídeo, A para audio, P para fotos).

```
* "A" for audio    (eg. media_dir=A,/var/lib/minidlna/music)
* "P" for pictures (eg. media_dir=P,/var/lib/minidlna/pictures)
* "V" for video    (eg. media_dir=V,/var/lib/minidlna/videos)
```

2. **Configuración del Nombre del Servidor (friendly_name):** Buscamos la línea de friendly_name y la descomentamos (quitando el #), asignándole el nombre que veremos en los clientes:



```
# Name that the DLNA server presents to clients.
# Defaults to "hostname: username".
friendly_name= Servidor DLNA PARP204
```

Una vez realizado estos pasos, guardamos los cambios y salimos del editor.

A diferencia de Jellyfin, MiniDLNA requiere que sus puertos específicos (estén abiertos en el *Firewall* de Ubuntu para ser detectado y para la transmisión de datos.

Para abrir los puertos necesarios ejecutamos los siguientes comandos:

- `sudo ufw allow 8200/tcp.`
- `sudo ufw allow 1900/udp.`
- `sudo ufw reload.`

```
avb99@avb99-VirtualBox:~$ sudo ufw allow 8200/tcp
Reglas actualizadas
Reglas actualizadas (v6)
avb99@avb99-VirtualBox:~$ sudo ufw allow 1900/udp
Reglas actualizadas
Reglas actualizadas (v6)
avb99@avb99-VirtualBox:~$ sudo ufw reload
```

Una vez realizados estos comandos, reiniciamos el servidor MiniDLNA para que aplique la nueva configuración y se registre en la red con los puertos abiertos. Para ello ejecutamos el siguiente comando:

- `sudo systemctl restart minidlna.`

A continuación, verificamos que el servicio está corriendo correctamente y lo realizamos con el siguiente comando:

- `sudo systemctl status minidlna`

INSTALACIÓN Y CONFIGURACIÓN BÁSICAS DE UN SERVIDOR MULTIMEDIA

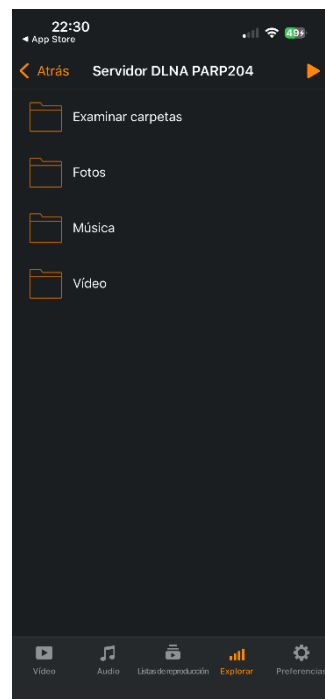
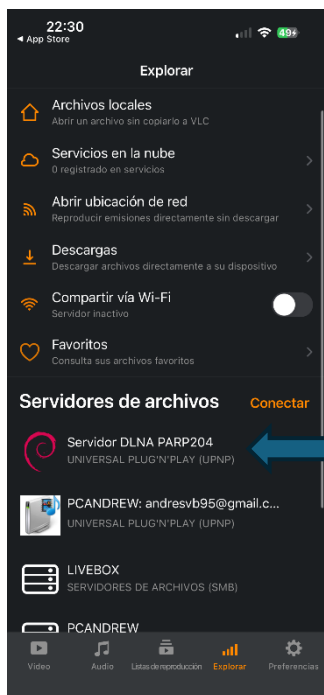
```
El cortafuegos no está activado (omitiedo recarga)
avb99@avb99-VirtualBox: $ sudo systemctl restart minidlna
avb99@avb99-VirtualBox: $ sudo systemctl status minidlna
● minidlna.service - MiniDLNA lightweight DLNA/UPnP-AV server
   Loaded: loaded (/usr/lib/systemd/system/minidlna.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-11-24 22:27:09 CET; 18s ago
     Docs: man:minidlnad(1)
           man:minidlna.conf(5)
   Main PID: 2219 (minidlnad)
    Tasks: 2 (limit: 5860)
   Memory: 9.3M (peak: 11.7M)
      CPU: 29ms
   CGroup: /system.slice/minidlna.service
           └─2219 /usr/sbin/minidlnad -f /etc/minidlna.conf -P /run/minidlna/minidlna.pid -S -r

nov 24 22:27:09 avb99-VirtualBox systemd[1]: Started minidlna.service - MiniDLNA lightweight DLNA/UPnP-AV server.
nov 24 22:27:09 avb99-VirtualBox minidlnad[2219]: parsing error file /etc/minidlna.conf line 22 : * "A" for audio (e
nov 24 22:27:09 avb99-VirtualBox minidlnad[2219]: parsing error file /etc/minidlna.conf line 23 : * "P" for pictures (e
nov 24 22:27:09 avb99-VirtualBox minidlnad[2219]: parsing error file /etc/minidlna.conf line 24 : * "V" for video (e
nov 24 22:27:09 avb99-VirtualBox minidlnad[2219]: minidlna.c:1163: warn: Starting MiniDLNA version 1.3.3.
nov 24 22:27:09 avb99-VirtualBox minidlnad[2219]: minidlna.c:1211: warn: HTTP listening on port 8200
nov 24 22:27:09 avb99-VirtualBox minidlnad[2220]: playlist.c:135: warn: Parsing playlists...
nov 24 22:27:09 avb99-VirtualBox minidlnad[2220]: playlist.c:269: warn: Finished parsing playlists.
lines 1-20/20 (END)
```

Ahora ya podemos verificar el funcionamiento del servidor en nuestro iPhone.

El protocolo DLNA no utiliza una dirección web, sino que se "anuncia" en la red local. Por ello, se utilizamos de nuevo la aplicación compatible VLC Media Player en el iPhone para la verificación.

Aquí, podemos ver que el servidor funciona correctamente en nuestro iPhone a través de la app:



CONCLUSIÓN

La práctica ha demostrado que la elección del servidor multimedia depende directamente del objetivo final.

En nuestro caso, se utilizaría “*Jellyfin*” para una experiencia de usuario rica y centralizada, y “*MiniDLNA*” para una solución de bajo consumo y alta compatibilidad básica en la red.

El mayor aprendizaje que he sacado de esta práctica es en la gestión de la capa de red para hacer accesibles los servicios virtuales al entorno físico, confirmando la solidez de las configuraciones implementadas.